齊魯理工學院

QILU INSTITUTE OF TECHNOLOGY

本科毕业设计（论文）

题　　目　基于 Python 的 Web 木材表面缺陷智能检测系统的设计与实现

姓　　名　张海娇　学号 231053011717

二级学院　计算机与信息工程学院

专　　业　计算机科学与技术

指导教师　黄浩英　职称　讲师

齐鲁理工学院教务处制

目 录

1 绪论 5
1.1 研究背景 5
1.2 研究现状 5
1.2.1 国外研究现状 5
1.2.2 国内研究现状 6
1.3 论文组织结构 6
2 关键技术 6
2.1 Python 6
2.2 Flask 7
2.3 YOLO 目标检测算法 7
2.4 SQLite 与 SQLAlchemy 7
3 系统分析 8
3.1 可行性研究 8
3.1.1 经济可行性 8
3.1.2 操作可行性 8
3.1.3 技术可行性 8
3.2 需求分析 8
3.3 系统功能分析 9
4 系统设计 9
4.1 系统结构设计 9
4.2 系统顺序图设计 10
4.2.1 登录顺序图 10
4.2.2 缺陷检测顺序图 11
4.2.3 历史追溯与报表顺序图 12
4.2.4 销售出库顺序图 12
4.3 数据库设计 13
4.3.1 数据库 E-R 图设计 13
4.3.2 数据库表设计 14
5 系统实现 15
5.1 用户功能模块的实现 15
5.1.1 用户登录界面 15
5.1.2 检测历史查询界面 17
5.1.3 日志报表查看界面 18
5.1.4 销售与库存管理界面 20
5.1.5 缺陷图谱与帮助中心界面 21
5.2 管理员功能模块的实现 22
5.2.1 用户管理界面 22

5.2.2 系统设置与模型切换界面 23
5.3 算法检测模块的实现 24
5.3.1 数据获取与预处理 24
5.3.2 检测模型设计与训练 25
5.3.3 系统检测与可视化分析 26
6 系统测试 28
6.1 测试目的 28
6.2 测试环境 28
6.3 测试用例 28
6.3.1 正常功能测试截图 29
6.3.2 异常功能测试截图 30
6.4 测试结论 31
7 总结与展望 31
参考文献 31
致谢 32

基于 Python 的 Web 木材表面缺陷智能检测系统的设计与实现

计算机科学与技术专业

指导教师

摘要：为解决传统木材表面缺陷检测过程中人工识别效率较低、检测结果不易保存、质量分级与库存管理衔接不够紧密等问题，本文设计并实现了一套基于 Python 的 Web 木材表面缺陷智能检测系统。系统以 Python 作为主要开发语言，采用 Flask 框架完成后端业务处理，使用 SQLite 数据库存储用户信息、检测记录、工作日志、库存数据和销售记录，并结合 YOLO 目标检测算法完成木材表面缺陷识别。在算法实现方面，系统通过加载训练后的权重文件对上传图片进行推理，获得缺陷类别、置信度和检测结果图，再根据裂纹、死节、活节等缺陷类型完成板材等级判定。在功能实现方面，系统完成了用户登录、图片上传检测、摄像头采集检测、检测历史查询、日志报表统计、Excel 数据导出、库存联动、销售出库、缺陷图谱展示以及管理员用户管理、参数设置和模型切换等功能。测试结果表明，该系统能够较好地完成木材表面缺陷检测与业务数据管理工作，具有一定的实用价值和教学演示价值。
关键词：Python 木材缺陷检测 YOLO Flask Web 系统 库存销售联动

Design and Implementation of a Python-based Web Wood Surface Defect Detection System

Computer Science and Technology

Tutor

Abstract: In order to solve the problems of low efficiency in manual wood surface defect inspection, difficulty in record storage, and weak connection between quality grading and inventory management, this paper designs and implements a Python-based web intelligent detection system for wood surface defects. The system uses Python as the main development language, Flask as the backend framework, SQLite as the database, and YOLO as the object detection algorithm. The trained model is used to infer uploaded wood images and output defect categories, confidence values and result images. According to the detected defects such as cracks, dead knots and live knots, the system further completes timber grade judgement. The implemented functions include user login, image upload detection, camera-based detection, detection history query, log report statistics, Excel export, inventory linkage, sales outbound processing, defect knowledge display, administrator user management, parameter configuration and model switching. The test results show that the system can complete wood surface defect detection and related business data management, and has practical value for teaching demonstration and graduation project application.
Keywords: Python; Wood Defect Detection; YOLO; Flask; Web System; Inventory and Sales

1 绪论

1.1 研究背景

随着家具制造、建筑装饰和板材加工行业的发展，木材质量检测在生产管理中的作用日益明显。木材表面常见的裂纹、死节、活节等缺陷会影响板材外观、力学性能和后续加工利用率，若不能及时识别和记录，容易造成分级不准确、库存统计混乱以及销售环节数据失真等问题。传统检测方式主要依赖人工观察，检测结果受人员经验、工作状态和现场环境影响较大，难以保证长期稳定性。因此，研究一种能够对木材表面缺陷进行自动识别、结果保存和等级管理的系统，具有较强的现实意义。

近年来，计算机视觉和深度学习技术在工业检测领域应用广泛，目标检测算法能够从图像中定位目标区域并输出类别信息，为木材缺陷检测提供了新的技术路径。相比人工检测方式，基于图像识别的检测方法具有速度快、结果可保存、便于统计分析等特点。其中，YOLO 系列算法属于典型的单阶段目标检测方法，在检测速度和工程部署方面具有一定优势，适合应用于木材表面缺陷识别系统。

基于上述背景，本文围绕木材表面缺陷检测与业务管理需求，设计并实现 Web 木材表面缺陷智能检测系统。系统不仅完成缺陷图像识别，还进一步将检测结果与板材等级、检测历史、日志统计、库存变化和销售出库等业务流程结合起来，从而形成从图像采集、模型检测、等级判定到数据管理的完整流程。通过该系统的设计与实现，可以为木材缺陷检测信息化管理提供一种可行的实现方案。

1.2 研究现状

1.2.1 国外研究现状

国外在木材缺陷检测和工业视觉识别方面研究较早。早期研究主要依靠图像纹理特征、边缘特征和颜色特征对木材表面进行分析，再结合支持向量机、决策树等传统机器学习方法完成缺陷分类。随着深度学习技术的发展，卷积神经网络、目标检测网络和图像分割模型逐渐应用于工业缺陷检测领域，研究重点也由单一特征提取转向端到端识别和实时检测。

在工程应用方面，国外部分木材加工、金属制造、纺织和电子生产企业已经将机器视觉系统嵌入生产线，用于质量分级、异常报警和统计分析。相关研究表明，缺陷检测系统的应用价值不仅体现在识别准确率上，还体现在检测稳定性、设备适应性以及与生产管理系统的结合能力上。因此，国外研究为木材缺陷检测系统的工程化应用提供了较多参考。

1.2.2 国内研究现状

国内关于木材缺陷识别、质量分级和智能制造系统的研究也在不断发展。近年来，随着 Python、OpenCV、PyTorch 等工具的普及，越来越多研究将图像处理、机器学习和深度学习算法应用于木材表面缺陷检测。部分研究通过改进目标检测模型提高缺陷识别准确率，部分研究则从检测系统开发角度出发，实现图片上传、模型推理和结果展示等功能。

总体来看，国内相关研究已经为木材缺陷检测提供了较好的算法基础和系统实现思路。但在课程设计和毕业设计类系统中，部分项目仍主要停留在上传图片、调用模型和显示检测结果层面，对检测历史保存、角色权限管理、库存联动、销售出库和报表统计等业务环节考虑不足。本文在已有研究基础上，结合 Web 系统开发方法，将木材缺陷检测结果进一步应用到等级判定和业务管理中，以提高系统的完整性和实用性。

1.3 论文组织结构

本文共分为七章，各章内容安排如下。

第 1 章为绪论，主要介绍课题研究背景、国内外研究现状以及论文整体结构。

第 2 章为关键技术，主要介绍系统开发过程中使用的 Python、Flask、YOLO 目标检测算法、SQLite 与 SQLAlchemy 等相关技术。

第 3 章为系统分析，主要从可行性、需求、功能以及性能与安全等方面对系统进行分析。

第 4 章为系统设计，主要完成系统结构设计、系统顺序图设计和数据库设计。

第 5 章为系统实现，主要介绍用户功能模块、管理员功能模块和算法检测模块的具体实现过程。

第 6 章为系统测试，主要说明测试目的、测试环境、测试用例和测试结论。第 7 章为总结与展望，对系统实现成果、存在不足及进一步研究方向进行总结。

2 关键技术

2.1 Python

Python 是一种解释型高级程序设计语言，具有语法简洁、类库丰富、开发效率高等特点。在 Web 开发、数据处理、人工智能和自动化脚本编写等领域，Python 均具有较广泛的应用。本系统采用 Python 作为主要开发语言，主要用于后端路由处理、数据库操作、YOLO 模型调用、图像文件处理以及报表数据导出。通过统一的 Python 开发环境，可以降低系统联调难度，提高开发效率。

在本系统中，Python 作为统一的开发语言，承担了业务处理与算法调用两方面任务。后端程序通过 Python 调用 Flask 框架完成页面请求响应，利用 SQLAlchemy 实现数据库操作，并借助 Ultralytics 接口加载目标检测模型完成木材表面缺陷识别。因此，Python 为 Web 服务、数据库管理和检测算法集成提供了统一的实现基础。

2.2 Flask

Flask 是一个轻量级 Web 应用框架，具有结构灵活、路由清晰、扩展方便等特点。与大型 Web 框架相比，Flask 对项目结构约束较少，适合中小型系统和课程设计项目快速开发。本系统使用 Flask 构建后端服务，通过路由函数分别实现登录认证、缺陷检测、历史记录查询、日志报表、销售管理和后台设置等功能。

同时，系统结合 Flask-Login 完成用户身份认证和会话管理。普通用户与管理员虽然使用同一套系统入口，但系统会根据用户角色控制可访问页面和可执行操作，从而保证系统具备基本的权限管理能力。

2.3 YOLO 目标检测算法

YOLO 是一种典型的单阶段目标检测算法，其基本思想是在一次前向推理过程中同时完成目标位置预测和类别判断。该类算法具有检测速度较快、部署流程相对简洁等特点，适合应用于对实时性有一定要求的工业视觉检测任务。本系统采用训练后的 YOLO 权重文件对木材图像进行推理，输出缺陷类别、置信度以及带有检测框的结果图。

在系统实现过程中，模型输出结果并未直接作为最终业务结果展示。系统首先将模型识别出的英文类别映射为中文缺陷名称，再依据缺陷类型完成 A 级、B 级和 C 级板材等级判定。该处理方式提高了检测结果的可读性，同时为库存统计与销售管理提供了统一的数据表达形式。

2.4 SQLite 与 SQLAlchemy

SQLite 是一种轻量级关系型数据库，具有部署简单、无需单独数据库服务、文件化存储等特点。对于毕业设计和单机演示类系统而言，SQLite 能够满足基本数据存储需求，同时降低系统安装和运行成本。本系统使用 SQLite 保存用户信息、检测历史、工作日志、销售记录、系统参数和库存数据。

SQLAlchemy 是 Python 中常用的对象关系映射工具，能够将数据库表映射为 Python 类，便于开发者通过对象方式完成数据库操作。本系统通过 SQLAlchemy 建立用户表、检测历史表、工作日志表、销售记录表、系统设置表和库存表，为检测记录追溯、报表统计、数据导出和库存联动提供数据支撑。

3 系统分析

3.1 可行性研究

3.1.1 经济可行性

从经济角度看，本系统的主要投入集中在开发时间、测试图片准备和运行环境调试等方面，而不涉及高额商业软件采购成本。项目所使用的 Python、Flask、Ultralytics、OpenCV、Bootstrap 和 Chart.js 等技术组件获取便捷，能够较好地满足毕业设计阶段的开发与演示需求。

3.1.2 操作可行性

从系统使用流程来看，系统操作路径较为清晰。登录后，常用功能入口集中显示在左侧导航区域，用户能够较快进入检测、历史记录、日志报表和销售管理等页面。检测页面支持本地上传与摄像头采集两种方式，管理员相关配置集中在系统设置页面中，能够满足系统日常使用与功能验证需求。

3.1.3 技术可行性

技术可行性主要体现在两个方面。一方面，项目所采用的各类技术组件较为成熟，相关资料较为丰富，便于开发过程中进行问题定位与调试。另一方面，系统已经能够完成从登录、检测到库存更新和销售记录写入的完整业务流程，说明现有技术路线具备实现条件。

此外，系统核心功能主要由后端业务处理模块与图像检测模块共同完成，这一结构安排使系统在模型替换、业务扩展和部署优化等方面具有较为明确的改进基础。

3.2 需求分析

结合系统实现情况，本文将系统用户划分为普通用户和管理员两类角色。普通用户主要负责登录、缺陷检测、历史记录查询和报表浏览；管理员在此基础上进一步承担用户维护、参数调整和模型切换等管理任务。上述角色划分已经在系统实现中得到落实，并形成了较为明确的功能边界与权限控制基础。

在检测业务上，系统需要支持单张与多张图片检测，支持浏览器摄像头采集，支持输出缺陷类别、置信度和板材等级，并将结果图保存在服务器指定目录。在业务管理上，系统需要记录每次检测的文件名、时间、操作人、结果图路径和等级信息，能够按照缺陷类型进行搜索和分页查询。在报表分析上，系统需要统计今日检测量、累计工作时长、缺陷类别占比和近 7 天检测趋势，并支持导出 Excel。

在流程联动方面，系统还需要在检测完成后自动更新对应等级库存，在销售页面创建订单并自动扣减库存，从而形成从检测到入库再到出库的闭环。帮助中心则用于展示常见缺陷图谱和等级判定规则，满足新用户学习需求。

3.3 系统功能分析

从系统实现结构来看，本系统可划分为页面展示层、业务控制层、模型推理层和数据存储层。这种分层方式能够较好地对应系统实际实现结构，有助于提高论文分析内容与项目实现之间的一致性。

从角色角度分析，普通用户的核心功能包括：登录认证、木材缺陷检测、历史记录查询、日志报表查看、销售信息查看、帮助中心学习和个人密码修改。管理员在此基础上新增用户管理、系统参数调节和模型文件切换功能。其中，检测模块是系统核心；历史、报表和销售模块体现系统的业务闭环；帮助中心和后台配置则增强了系统的实用性和可维护性。

若按照功能入口进一步细分，系统主要包括登录、仪表盘、缺陷检测、历史记录、日志报表、数据导出、销售管理、帮助中心、用户管理和系统设置等功能页面。各功能页面与后端业务处理模块相互对应，形成了较为清晰的页面组织结构，从而便于系统维护与功能扩展。

4 系统设计

4.1 系统结构设计

本系统采用 B/S 架构进行设计，用户通过浏览器访问系统页面，服务器端负责业务处理、模型推理和数据库操作。该结构能够降低客户端部署要求，便于用户在局域网或本地环境中直接访问系统。系统总体结构如图 4-1 所示。

从系统组成来看，前端页面负责图片选择、结果展示、图表显示和表单提交；后端服务负责接收请求、调用检测模型、处理业务逻辑并返回结果；数据库负责保存用户、检测记录、日志、库存和销售等数据；算法模块负责完成木材表面缺陷识别。各部分相互配合，共同完成系统功能。

系统启动时会创建上传目录和结果目录，并完成数据库表初始化。当系统中不存在默认管理员账号或初始库存数据时，程序会自动写入基础数据，以保证系统首次运行后能够进行登录、检测和业务演示。

系统页面采用统一布局方式进行设计。登录后的页面通过公共模板组织侧边导航、用户信息、提示信息和功能入口，各业务页面在统一布局基础上展示具体内容。该设计有利于保持页面风格一致，并为系统维护与功能扩展提供便利。

系统业务层主要由后端路由函数组成。不同路由分别对应登录、检测、历史记录、日志报表、销售管理、用户管理和系统设置等业务请求，在接收前端数据后完成权限判断、数据处理、模型调用和数据库提交。

系统数据流主要包括检测数据流、统计数据流和销售数据流。检测数据流从图片上传开始，经模型推理、等级判定后写入检测历史并更新库存；统计数据流从检测历史和工作日志中提取数据，用于生成报表和趋势图；销售数据流根据用户提交的订单信息扣减库存并保存销售记录。

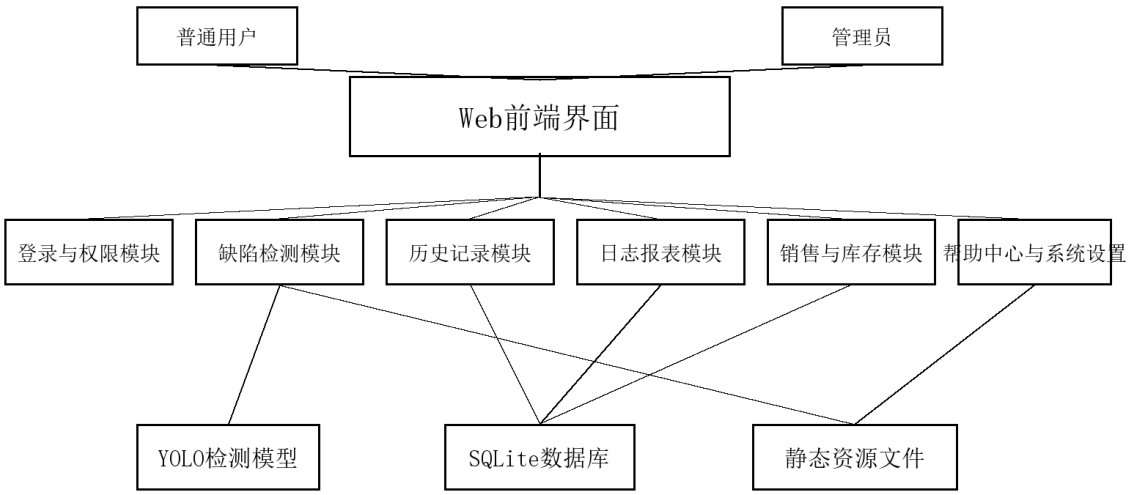

图 4-1 系统总体结构图

4.2 系统顺序图设计

4.2.1 登录顺序图

登录顺序图用于描述用户登录系统时前端页面、后端服务和数据库之间的交互过程。用户在登录页面输入账号、密码并选择登录身份后，系统首先查询用户信息，然后校验密码和角色是否匹配。验证通过后写入登录状态并记录工作日志，验证失败时返回错误提示。登录顺序图如图 4-2 所示。

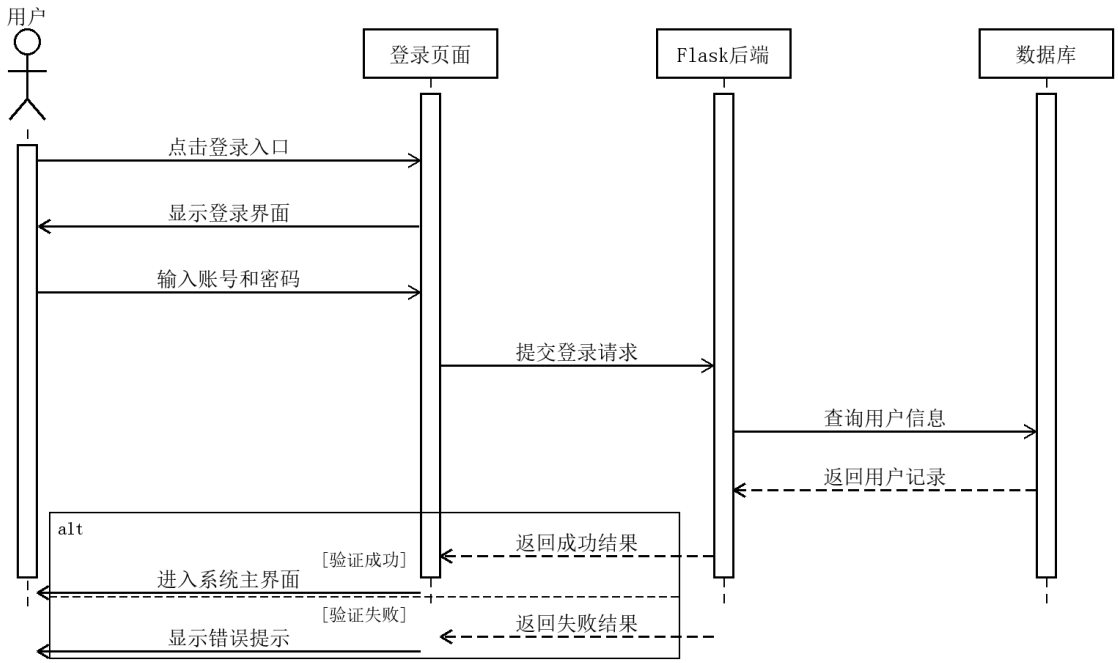

图 4-2 登录顺序图

4.2.2 缺陷检测顺序图

缺陷检测顺序图用于说明图片上传、模型推理、结果处理和数据保存之间的调用关系。用户提交图片后，后端保存原始图片并读取当前检测参数，随后调用 YOLO 模型完成推理。系统根据模型输出结果生成检测图、缺陷类别、置信度和等级信息，并将检测结果写入数据库。缺陷检测顺序图如图 4-3 所示。

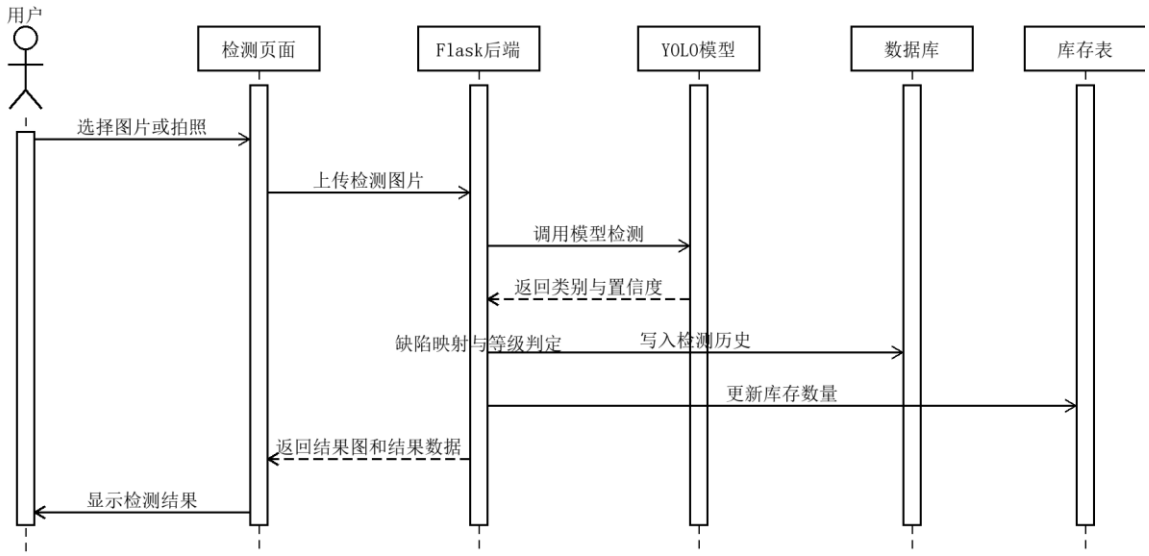

图 4-3 缺陷检测顺序图

4.2.3 历史追溯与报表顺序图

历史追溯与报表顺序图用于描述系统对检测数据进行查询、统计和展示的过程。用户进入历史记录或日志报表页面后，系统根据当前用户身份确定数据查询范围，随后完成分页查询、缺陷类型筛选、统计汇总和图表展示。该流程能够支持检测记录追溯和运行数据分析。历史追溯与报表顺序图如图 4-4 所示。

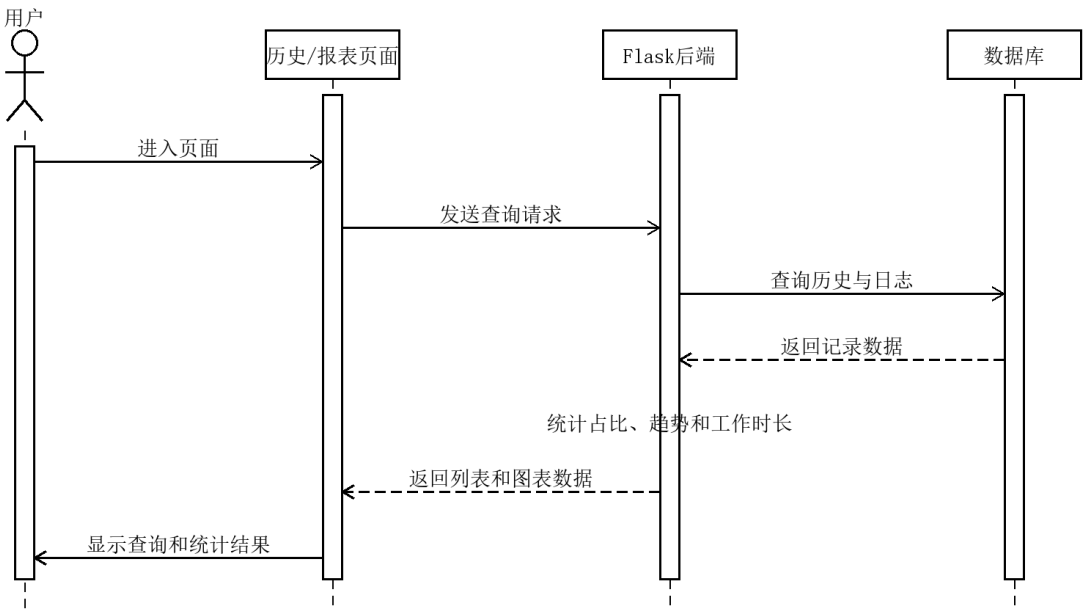

图 4-4 历史与报表顺序图

4.2.4 销售出库顺序图

销售出库顺序图用于说明销售订单提交与库存扣减之间的业务流程。用户在销售页面填写客户名称、板材等级、数量和金额后，系统首先查询对应等级库存数量。当库存满足出库条件时，系统扣减库存并保存销售记录；当库存不足时，系统返回提示信息并终止出库操作。销售出库顺序图如图 4-5 所示。

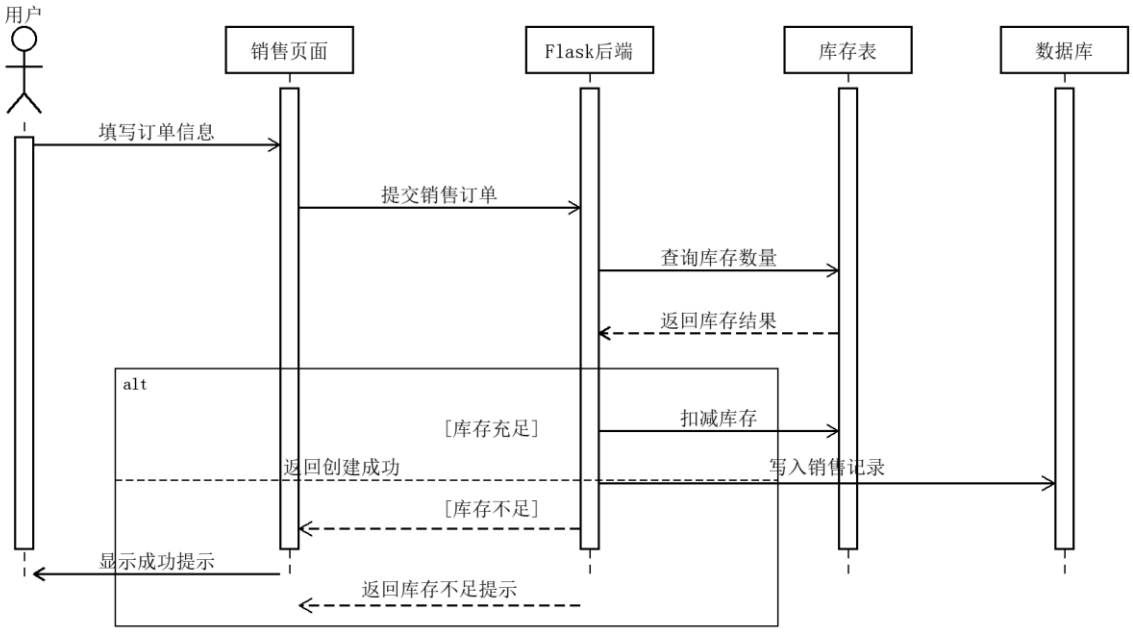

图 4-5 销售出库顺序图

4.3 数据库设计

4.3.1 数据库 E-R 图设计

数据库 E-R 图用于表示系统中主要实体及其联系。本系统围绕用户、检测历史、工作日志、销售记录和库存等数据对象展开设计，其中用户与检测历史、工作日志存在关联关系，库存与销售记录通过板材等级形成业务联系。主要实体属性图如图 4-6 所示。

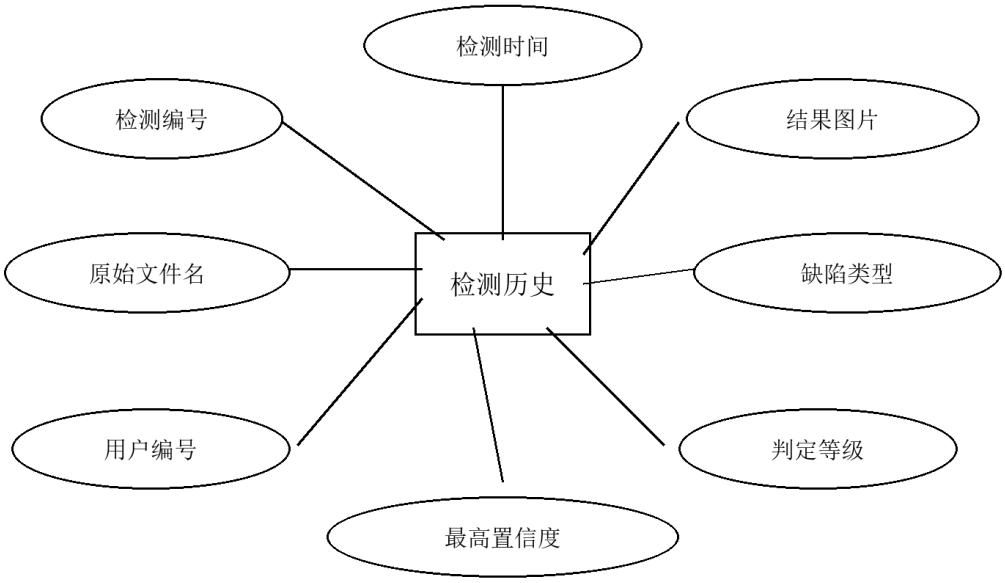

图 4-6 主要实体属性图

根据系统业务流程，用户完成登录后可以产生检测记录和工作日志；检测结果经过等级判定后会影响库存数量；销售出库操作会读取库存并生成销售记录。系统 E-R 图如图 4-7 所示。

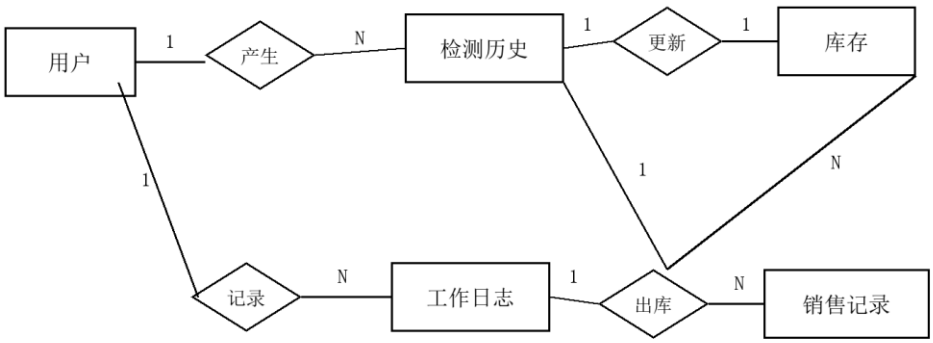

图 4-7 系统 E-R 图

4.3.2 数据库表设计

根据系统功能需求和 E-R 图设计，数据库主要包括用户表、检测历史表、工作日志表、销售记录表、系统设置表和库存表。各表分别承担账号管理、检测结果保存、工作时间记录、销售业务保存、参数配置和库存统计等功能。数据库表结构设计如下。

表 4-1 用户表 User

字段名	类型	说明
id	Integer	主键
username	String(150)	用户名，唯一
password	String(150)	密码哈希
role	String(20)	角色，user/admin

表 4-2 检测历史表 DetectionHistory

字段名	类型	说明
id	Integer	主键
filename	String(300)	原始文件名
result_image	String(300)	结果图文件名

upload_date	DateTime	上传检测时间
defect_type	String(100)	缺陷类型
confidence	Float	最高置信度
user_id	Integer	关联用户 ID
grade	String(10)	等级结果

表 4-3 工作日志表 WorkLog

字段名	类型	说明
id	Integer	主键
user_id	Integer	操作用户 ID
login_time	DateTime	登录时间
logout_time	DateTime	退出时间

表 4-4 销售记录表 SalesRecord

字段名	类型	说明
id	Integer	主键
customer_name	String(100)	客户名称
product_grade	String(50)	销售等级
quantity	Integer	数量
total_price	Float	订单总价
sale_date	DateTime	销售时间

表 4-5 系统设置表 SystemSettings

字段名	类型	说明
id	Integer	主键
key_name	String(50)	设置键名
value	String(200)	设置值

表 4-6 库存表 Inventory

字段名	类型	说明
grade	String(10)	等级主键
count	Integer	库存数量

5 系统实现

5.1 用户功能模块的实现

5.1.1 用户登录界面

登录模块采用统一登录页面设计，普通用户与管理员通过不同标签页进入各自身份入口。后端在接收登录请求后，会结合账号信息与入口类型完成角色匹配校验，从而保证不同身份

用户访问权限的正确性。用户退出登录时，系统还会同步补写工作日志结束时间，为工作时长统计提供数据基础。

图 5-1 登录界面截图

部分关键代码如下：

```
# app.py
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')
        login_role = request.form.get('login_role')
        user = User.query.filter_by(username=username).first()

        if user and check_password_hash(user.password, password):
            if user.role != login_role:
                flash('身份不匹配，请切换到对应标签页登录。', 'warning')
                return redirect(url_for('login'))
            login_user(user)
            return redirect(url_for('dashboard'))
    return render_template('login.html')
```

5.1.2 检测历史查询界面

历史记录页面的实现重点在于检测结果的可查询性与可追溯性。页面集中展示结果图、文件名、等级、缺陷类型和检测时间等信息，并支持按照缺陷关键词进行筛选。考虑到系统运行过程中会持续积累检测记录，因此页面采用分页方式控制单页展示数量，以提升浏览效率。

在删除记录操作中，后端增加了权限控制逻辑。管理员可以查看并管理全量检测数据，普通用户仅能操作本人产生的记录，该设计能够保证多用户场景下的数据访问边界，并提高系统操作的安全性。

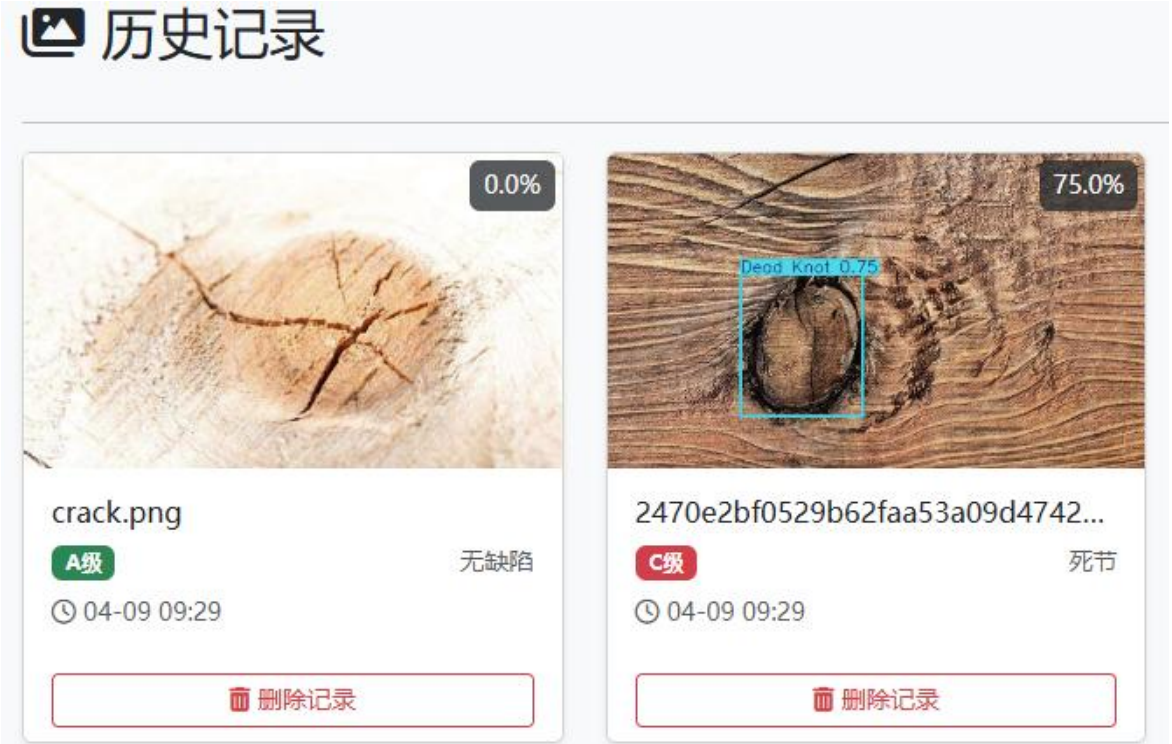

图 5-3 历史记录页面截图

部分关键代码如下：

```
# app.py
@app.route('/history', methods=['GET', 'POST'])
@login_required
def history():
    page = request.args.get('page', 1, type=int)
    search = request.args.get('search', '')
    query = DetectionHistory.query
    if current_user.role != 'admin':
```

```
    query = query.filter_by(user_id=current_user.id)
if search:
    query = query.filter(DetectionHistory.defect_type.contains(search))
records = query.order_by(
    DetectionHistory.upload_date.desc()
).paginate(page=page, per_page=12)
```

5.1.3 日志报表查看界面

日志与报表模块的主要作用是对系统已保存的检测数据进行统计整理与可视化展示。系统能够计算今日检测量、缺陷类别占比和近 7 天检测趋势，并通过前端图表组件完成可视化呈现，从而为运行状态分析与结果汇总提供支持。

数据导出功能通过 pandas 和 openpyxl 对检测记录进行整理，并生成 Excel 文件供浏览器下载。该功能能够将系统中的结构化数据转换为便于归档与统计分析的表格文件，从而提升检测结果的数据利用价值。

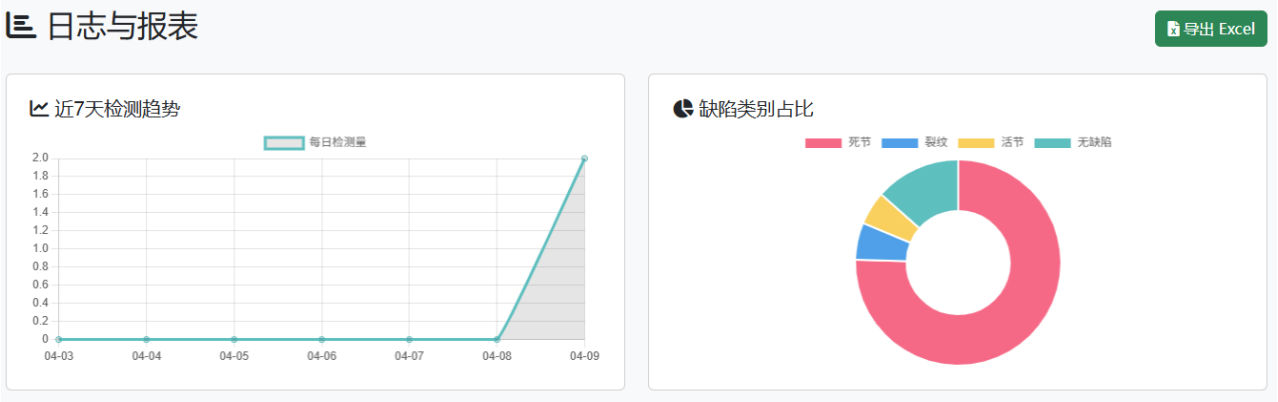

图 5-4 日志与报表页面截图

部分关键代码如下：

```
# app.py
@app.route('/logs')
@login_required
def logs():
    today_count = hist_query.filter(
        func.date(DetectionHistory.upload_date) == date.today()
    ).count()
    stats_data = [[k, v] for k, v in counter.items()]
    trend_data = []
    for i in range(6, -1, -1):
```

```
    target_date = date.today() - timedelta(days=i)
    count = hist_query.filter(
        func.date(DetectionHistory.upload_date) == target_date
    ).count()
    trend_data.append(count)
```

工作时长统计来源于 `WorkLog` 表。用户每次登录时系统都会新增一条日志记录，退出时再补上离开时间。报表页在统计累计工作时长时，会把已经结束的记录和当前仍处于登录状态的记录一起计算，所以页面上显示的是一个动态变化的结果，而不是固定写死的数字。

该统计逻辑在实现上相对清晰，但能够使系统记录内容从单纯的检测次数扩展到用户工作时长信息，从而提升系统日志数据的完整性，并为报表分析提供更多维度的参考。

图 5-5 工作时长统计截图

部分关键代码如下：

```
# app.py
total_seconds = 0
for l in log_query.filter(WorkLog.logout_time != None).all():
    total_seconds += (l.logout_time - l.login_time).total_seconds()

if session.get('current_log_id'):
```

```
    curr = db.session.get(WorkLog, session['current_log_id'])
    if curr:
        total_seconds += (datetime.now() - curr.login_time).total_seconds()

work_time_str = (
    f"{int(total_seconds // 3600)}小时 "
    f"{int((total_seconds % 3600) // 60)}分钟"
)
```

5.1.4 销售与库存管理界面

销售与库存管理页面在业务设计上与检测模块保持紧密联动。页面上方展示 A、B、C 三类板材库存数量，下方提供销售订单录入表单。用户提交订单后，系统会优先校验当前等级库存是否充足，仅在库存满足条件时执行出库并写入销售记录，从而保证销售数据与库存数据的一致性。

图 5-6 销售与库存页面截图

部分关键代码如下：

```
# app.py
@app.route('/sales', methods=['GET', 'POST'])
@login_required
def sales():
    if request.method == 'POST':
        grade = request.form.get('grade')
```

```
    qty = int(request.form.get('quantity'))
    inv = db.session.get(Inventory, grade)
    if not inv or inv.count < qty:
        flash(f'{grade} 库存不足！', 'danger')
    else:
        inv.count -= qty
        sale = SalesRecord(customer_name=request.form.get('customer'),
                           product_grade=grade, quantity=qty)
        db.session.add(sale)
```

5.1.5 缺陷图谱与帮助中心界面

帮助中心页面展示了活节、死节和裂纹三类典型样张，并给出相应的定义说明、外观特征和等级判定依据。该页面能够辅助用户理解不同缺陷之间的差异，并提高检测结果说明的直观程度。

图 5-7 帮助中心页面截图

部分关键代码如下：

```
# app.py / templates/help.html
@app.route('/help')
@login_required
def help_center():
```

```
    return render_template('help.html')

<img src="{{ url_for('static', filename='img/live_knot.png') }}"
    class="card-img-top" alt="活节样张">
<img src="{{ url_for('static', filename='img/dead_knot.png') }}"
    class="card-img-top" alt="死节样张">
<img src="{{ url_for('static', filename='img/crack.png') }}"
    class="card-img-top" alt="裂纹样张">
```

5.2 管理员功能模块的实现

5.2.1 用户管理界面

用户管理模块主要面向管理员使用，系统支持新增和删除用户账号，并在创建用户时对密码进行哈希处理。该模块实现了系统基础的账号维护功能，为权限控制和多用户使用提供了保障。

图 5-8 用户管理页面截图

部分关键代码如下：

```
# app.py
@app.route('/admin/users', methods=['GET', 'POST'])
@login_required
def admin_users():
    if request.method == 'POST':
        action = request.form.get('action')
        if action == 'add':
            new_user = User(
                username=request.form.get('username'),
                password=generate_password_hash(request.form.get('password')),
```

```
        role=request.form.get('role')
    )
    db.session.add(new_user)
    db.session.commit()
```

5.2.2 系统设置与模型切换界面

系统设置页面主要提供检测阈值与模型文件两类可调参数。检测阈值会影响模型推理时目标框的保留条件，模型上传功能则使管理员能够在不修改程序代码的情况下替换新的模型权重文件，从而体现系统具备一定的可维护性与扩展性。

图 5-9 系统设置页面截图

部分关键代码如下：

```
# app.py
@app.route('/settings', methods=['GET', 'POST'])
@login_required
def settings():
    val = request.form.get('conf_threshold')
    s = SystemSettings.query.filter_by(key_name='conf_threshold').first()
```

```
    s.value = val

    f = request.files.get('model_file')
    if f and f.filename.endswith('.pt'):
        fname = secure_filename(f.filename)
        f.save(os.path.join(app.root_path, fname))
        m = SystemSettings.query.filter_by(key_name='current_model').first()
        m.value = fname
```

5.3 算法检测模块的实现

5.3.1 数据获取与预处理

检测页面是系统核心业务页面之一。系统将本地上传和摄像头采集两种方式整合在同一页面中，并通过选项卡进行切换，以提高检测操作的连续性。页面同时增加了本地图片预览功能，便于用户在提交检测前确认所选文件。

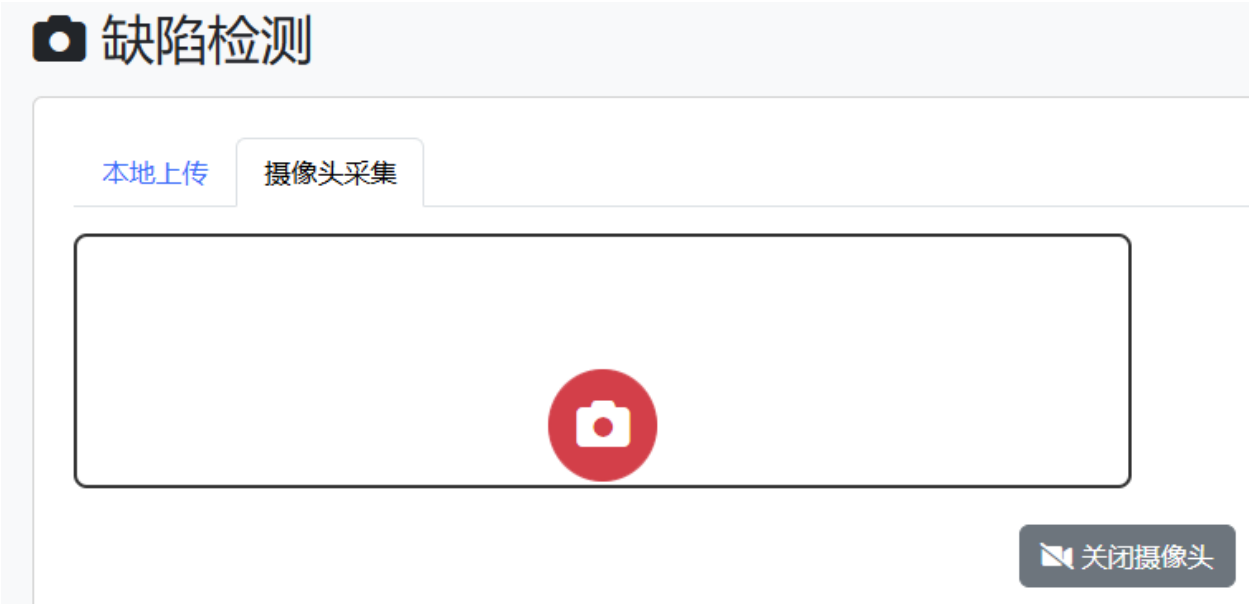

图 5-10 检测页面截图

部分关键代码如下：

```
# templates/detect.html
<ul class="nav nav-tabs mb-3" id="detectTab">
    <li class="nav-item">
        <button class="nav-link active" data-bs-target="#upload-pane"
                type="button" onclick="stopCamera()">本地上传</button>
    </li>
```

```
    <li class="nav-item">
        <button class="nav-link" data-bs-target="#camera-pane"
                type="button" onclick="initCamera()">摄像头采集</button>
    </li>
</ul>
<form id="uploadForm">
    <input class="form-control" type="file" id="images"
          name="images" multiple accept="image/*">
    <div class="row g-3 mb-3" id="previewArea"></div>
</form>
```

5.3.2 检测模型设计与训练

后端检测流程首先读取当前阈值和模型配置，再将原始图片保存至指定目录，随后调用检测函数执行模型推理。考虑到系统模型文件可能存在更新需求，因此后台设置页面保留了模型切换功能，后续检测时将优先使用管理员上传的新模型权重文件。

模型返回的原始类别为英文名称，系统在展示结果前会先将其映射为中文缺陷名称，再结合业务规则给出等级结果。与此同时，页面不仅展示缺陷类别，还同步显示结果图、置信度和等级判定信息，从而使单次检测过程能够同时输出较为完整的结果信息。

图 5-11 推理结果展示截图

部分关键代码如下：

```
# utils/detection.py
def process_image(image_path, save_dir, model_path, conf_threshold=0.25):
    model = load_model(model_path)
    img = cv2.imread(image_path)
```

```
    results = model(img, conf=float(conf_threshold))

    detected_objects = []
    max_conf = 0.0
    for r in results:
        res_plotted = r.plot()
        for box in r.boxes:
            cls_id = int(box.cls[0])
            detected_objects.append(model.names[cls_id])
            max_conf = max(max_conf, float(box.conf[0]))
    return {'defects': ','.join(list(set(detected_objects))),
            'confidence': round(max_conf, 2)}
```

5.3.3 系统检测与可视化分析

在本系统中，等级判定并不是结果展示阶段的附加信息，而是检测流程中的关键业务环节。系统默认将无缺陷板材判定为 A 级；识别到活节时判定为 B 级；若检测到裂纹或死节，则判定为 C 级。等级结果生成后，后端会继续更新库存表，从而使单次检测操作能够与库存管理流程有效衔接。

图 5-12 等级判定与库存联动截图

部分关键代码如下：

```
    # app.py
detected_grade = 'A级'
if res['is_clean']:
```

```
    detected_grade = 'A级'
else:
    for d in res['defects'].split(','):
        zh_name = DEFECT_MAPPING.get(d.strip(), d.strip())
        if zh_name in ['裂纹', '死节']:
            detected_grade = 'C级'
            break
        elif zh_name == '活节':
            detected_grade = 'B级'

inv = db.session.get(Inventory, detected_grade)
if inv:
    inv.count += 1
```

每完成一次检测后，系统都会将结果写入 `DetectionHistory` 表。保存内容包括原始文件名、结果图文件名、缺陷类型、最高置信度、操作用户和等级结果等字段。结果图片则单独保存在 `static/results` 目录中，前端页面通过静态资源路径进行访问。该设计实现了图片文件与结构化记录的分离存储，同时便于页面展示与报表统计进行关联。

该设计使历史追溯、图片预览和统计报表均建立在同一批检测数据基础之上，避免了重复整理数据的额外工作。

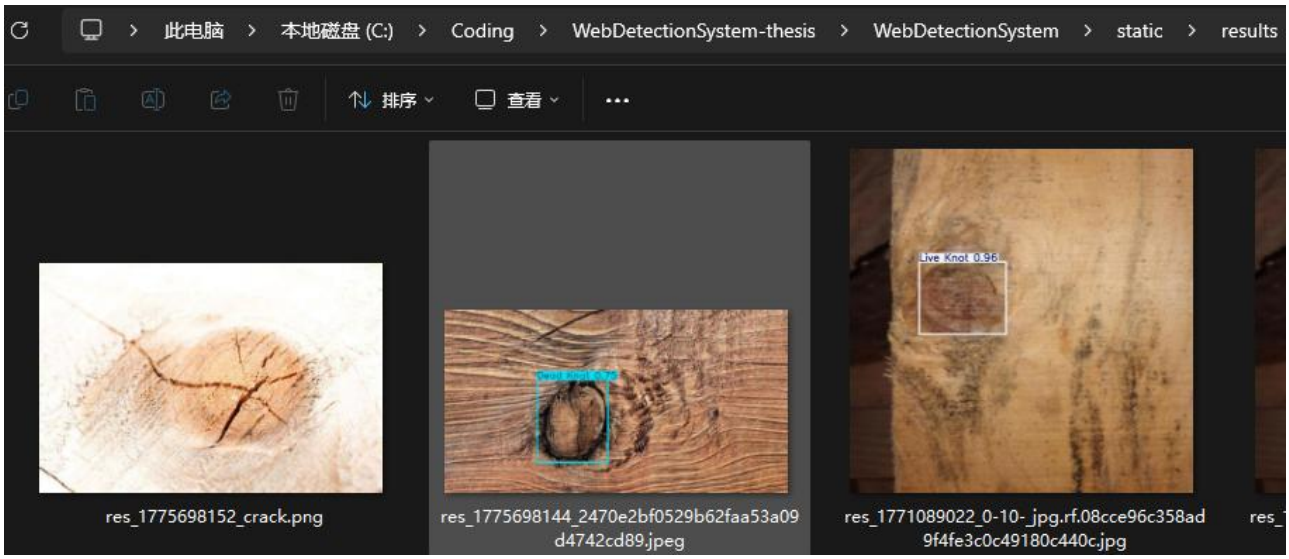

图 5-13 检测结果持久化截图

部分关键代码如下：

```
    # app.py
history = DetectionHistory(
    filename=filename,
    result_image=res['filepath'],
    defect_type=final_defect_str,
    confidence=res.get('confidence', 0.0),
    user_id=current_user.id,
    grade=detected_grade
)
db.session.add(history)
results.append(res)
db.session.commit()
```

6 系统测试

6.1 测试目的

系统测试的主要目的是验证木材表面缺陷智能检测系统在当前运行环境下能否稳定完成各项核心业务流程，并检查系统在正常输入和异常输入条件下的响应情况。本章测试重点包括用户登录、缺陷检测、检测记录保存、日志报表统计、Excel 数据导出、销售出库以及后台设置等功能。通过测试可以进一步说明系统是否满足毕业设计阶段的基本使用要求。

6.2 测试环境

本次测试所使用的环境与系统日常开发环境保持一致。测试平台为 Windows 操作系统，后端程序运行于本地 Python 虚拟环境中，Web 框架采用 Flask，数据库采用 SQLite，检测模块调用训练完成的模型权重文件进行推理。浏览器端用于验证登录、图片上传、摄像头采集、图表显示和销售管理等页面功能。

测试过程中分别以普通用户和管理员身份进入系统，按照实际业务流程对主要功能模块进行逐项验证。除正常测试外，还对未选择图片即提交检测、角色入口不匹配以及库存不足时继续提交订单等异常情况进行了检查，以观察系统是否能够给出明确提示。

6.3 测试用例

测试编号	测试内容	预期结果	测试结论
T1	普通用户登录	账号密码正确且角色匹配时成功进入仪表盘	通过
T2	管理员误用普通用户入口登录	系统提示身份不匹配并拒绝登录	通过

T3	上传单张图片检测	返回结果图、缺陷类别、等级和置信度	通过
T4	批量图片检测	多张图片依次返回检测结果并写入历史	通过
T5	摄像头拍照检测	拍照后成功提交并返回检测结果	通过
T6	按缺陷类型查询历史	历史记录支持关键字搜索与分页	通过
T7	导出 Excel 报表	浏览器下载包含检测记录的表格文件	通过
T8	库存不足时提交销售订单	系统阻止出库并提示当前库存	通过
T9	管理员上传新模型	保存成功并在后续检测中优先使用新模型	通过

表 6-1 系统功能测试用例

从测试结果来看，系统在正常使用场景下能够完成登录、检测、记录保存、统计报表、数据导出和库存销售联动等操作。在异常场景下，系统对角色不匹配、未选择图片、库存不足等情况也能够返回对应提示，说明系统具备基本的异常处理能力。

6.3.1 正常功能测试截图

在正常测试过程中，系统能够完成用户登录、图片检测、结果显示以及历史记录保存等功能。正常功能测试截图如图 6-1 所示。

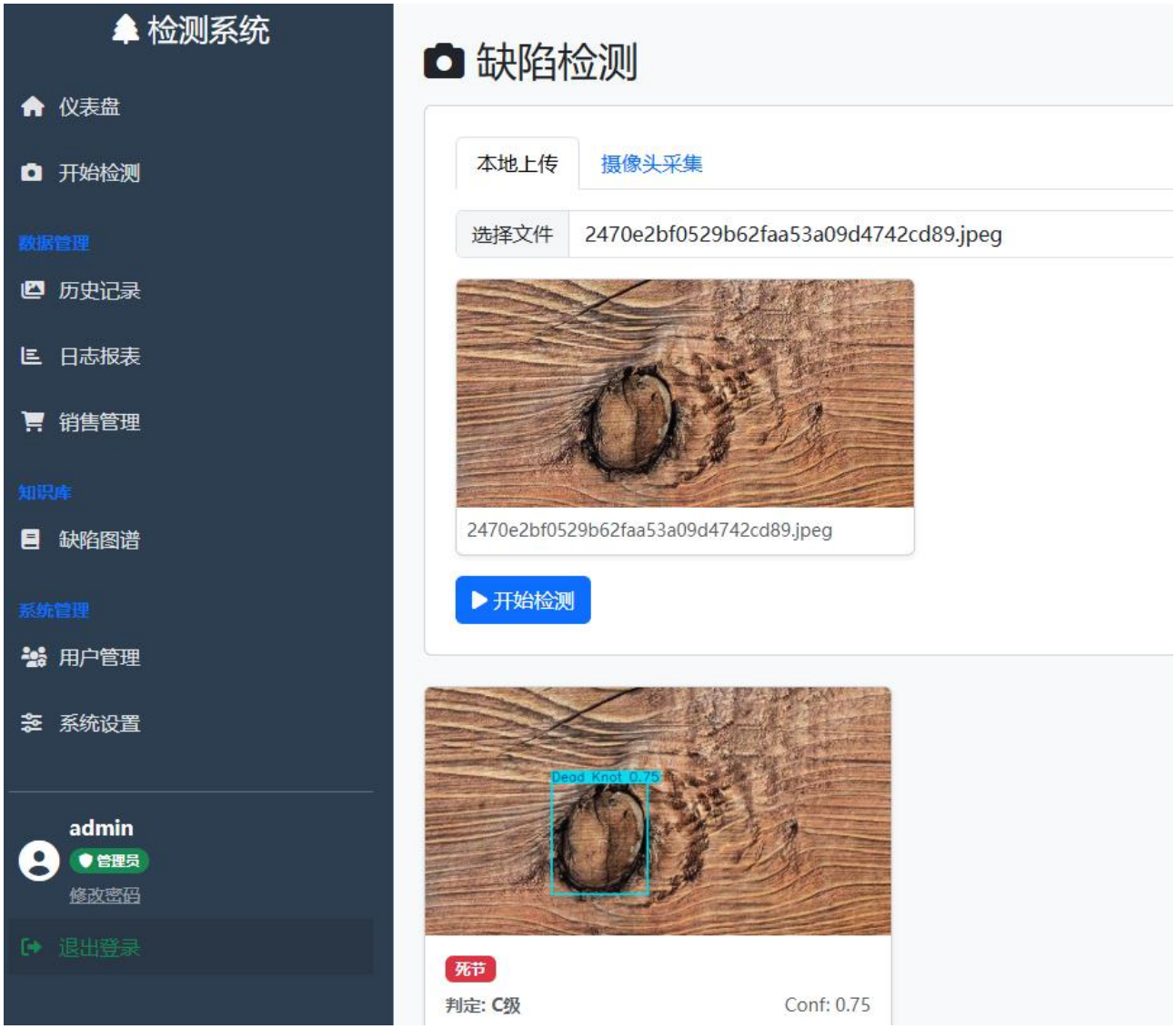

图 6-1 正常功能测试截图

6.3.2 异常功能测试截图

在异常测试过程中，系统能够对未选择图片即提交检测、身份不匹配登录以及库存不足出库等情况给出明确提示。异常功能测试截图如图 6-2 所示。

图 6-2 异常功能测试截图

6.4 测试结论

综合测试结果可知，系统已经能够较完整地实现论文前文设计的主要功能。系统能够完成木材表面缺陷图像检测、结果等级判定、检测历史追溯、日志报表统计、库存数量更新、销售出库和管理员后台管理等业务。整体运行过程较为稳定，能够满足系统功能验证与应用展示的基本要求。

但从测试情况看，系统仍存在进一步完善空间。例如，历史记录删除后尚未同步清理对应图片文件，系统数据库仍以 SQLite 单机部署为主，并且测试工作主要集中在功能验证层面，对高并发访问、异常恢复和长期运行稳定性的覆盖仍不够充分。这些问题可作为系统后续优化的重要方向。

7 总结与展望

总体而言，本文研究工作建立在已完成开发与调试的木材表面缺陷检测系统基础之上。论文中涉及的大部分页面、业务流程和数据结构均能够在系统实现中找到对应内容，因此论文分析内容与系统实现情况基本一致。

未来可从三个方面继续开展完善工作。其一，进一步补充数据集并优化模型，以提升系统在复杂木纹、光照变化以及多缺陷同时出现条件下的识别稳定性；其二，将现有单机运行方式逐步扩展为更适合多人使用的部署结构，以提升系统应用范围；其三，继续完善业务层功能，例如图片文件清理、权限颗粒度控制、日志审计和更完整的销售统计。在硬件与时间条件允许的前提下，系统还可进一步扩展至移动端采集或现场摄像设备应用场景。

参考文献

[1] Python Software Foundation. Python Documentation[EB/OL].
[2] Flask Documentation Team. Flask Documentation[EB/OL].
[3] SQLAlchemy Authors. SQLAlchemy Documentation[EB/OL].
[4] Ultralytics. Ultralytics YOLO Docs[EB/OL].
[5] OpenCV Team. OpenCV-Python Tutorials[EB/OL].
[6] Bootstrap Team. Bootstrap 5 Documentation[EB/OL].
[7] pandas development team. pandas Documentation[EB/OL].
[8] He K, Gkioxari G, Dollar P, et al. Mask R-CNN[C]. IEEE, 2017.
[9] Redmon J, Farhadi A. YOLOv3: An Incremental Improvement[EB/OL].
[10] Jocher G, Chaurasia A, Qiu J. YOLO by Ultralytics[EB/OL].
[11] 王耀南, 刘少创. 机器视觉原理与应用[M]. 北京: 机械工业出版社.
[12] 周志华. 机器学习[M]. 北京: 清华大学出版社.

致谢

本论文能够顺利完成，首先要感谢指导教师在选题确定、系统梳理和论文修改过程中给予的耐心指导。从需求整理、页面调整、数据库设计，到检测流程实现和正文修改，老师都提出了许多具体而宝贵的建议。同时，也感谢学院提供实验环境和学习平台，使本人能够将课程中接触到的 Python Web 开发、数据库应用和目标检测方法应用到实际项目之中。此外，还要感谢同学和家人在项目调试、资料整理与论文撰写期间给予的支持。通过本次毕业设计，本人对一个项目从系统实现到论文写作的完整过程有了更加清晰的认识，也为后续深入学习相关技术奠定了基础。